

ROYALE GOLD CASINO S.A.S.

INFORME DE ARQUITECTURA DE DATOS

Proyecto Final — Bases de Datos

Sistema de Gestión Integral para Casino Físico

Compatible con Oracle Database 21c XE / Oracle Live SQL

Nombre Completo	Código	Rol en el Proyecto
Carol Nicol Clavijo Bonilla	1028663315	Arquitecta de Datos y Modelado ER
Daniel Alejandro Marson Rosas	5671988	Desarrollador PL/SQL y Control ACID
Adrian David Bravo Montoya	1019902373	Líder de QA y Pruebas de Integración

Bases de Datos — 2025

SECCIÓN 1: PORTADA Y CONTEXTO CORPORATIVO

Nombre de la Empresa

Royale Gold Casino S.A.S. — *"Donde cada dato es una apuesta segura."*

Misión del Sistema

Royale Gold Casino S.A.S. opera una plataforma de gestión integral para casinos físicos que centraliza en una base de datos Oracle todos los procesos críticos del negocio: control de acceso de clientes, operación de mesas y máquinas tragamonedas, registro de transacciones de caja, programa de lealtad, gestión de torneos y cumplimiento regulatorio AML (Anti Lavado de Dinero). El sistema garantiza que cada operación sea atómica, trazable y conforme con la normativa de la Unidad de Información y Análisis Financiero (UIAF) de Colombia, eliminando la dependencia de lógica de negocio dispersa en aplicaciones externas.

Visión del Sistema

Ser la solución de referencia en gestión de datos para la industria del entretenimiento regulado en Colombia, ofreciendo una arquitectura de base de datos que soporte el crecimiento de la operación sin sacrificar integridad, seguridad ni trazabilidad.

Integrantes del Equipo

Nombre Completo	Código	Rol en el Proyecto
Carol Nicol Clavijo Bonilla	1028663315	Arquitecta de Datos y Modelado ER
Daniel Alejandro Marson Rosas	5671988	Desarrollador PL/SQL y Control ACID
Adrian David Bravo Montoya	1019902373	Líder de QA y Pruebas de Integración

SECCIÓN 2: LÓGICA DE NEGOCIO Y REQUERIMIENTOS

2.1 Reglas de Negocio del Sistema

Regla 1 — Mayoría de edad obligatoria: Ningún cliente menor de 18 años puede ser registrado ni realizar transacciones. Respaldada por la Ley 643 de 2001. Se implementa en el motor de base de datos para que no pueda ser eludida desde ninguna capa de aplicación. Objeto: **trg_cliente_edad_minima** (ORA-20001).

Regla 2 — Límite diario de gasto: Cada cliente tiene un tope máximo de gasto diario (default COP \$5,000,000). La suma de compras de fichas y depósitos no puede superar ese límite en un día calendario. El contador se resetea automáticamente con la primera transacción del nuevo día. Objeto: **trg_limite_diario** (ORA-20003).

Regla 3 — Autoexclusión total: Un cliente autoexcluido queda bloqueado permanentemente para cualquier transacción de gasto. El bloqueo opera antes de que la transacción llegue a procesarse en caja. Objeto: **trg_limite_diario** (ORA-20002), **sp_autoexcluir_cliente**.

Regla 4 — Alerta automática AML: Toda transacción $\geq$ COP \$10,000,000 genera automáticamente un incidente AML. El umbral corresponde al monto de reporte obligatorio de la UIAF Colombia. El proceso es completamente automático. Objeto: **trg_alerta_aml**.

Regla 5 — Integridad de mesas: No se puede iniciar una sesión en una mesa que no esté en estado ACTIVA. Mesas en MANTENIMIENTO o INACTIVA rechazan cualquier apertura de sesión. Objeto: **trg_validar_mesa_activa** (ORA-20004).

Regla 6 — Seguridad financiera: El personal de caja, croupiers y clientes no tienen acceso al RTP de las máquinas ni a los rangos de apuesta de las mesas. Esta información permitiría calcular estrategias que minimizan la ventaja de la casa. Objeto: **v_catalogo_publico**.

2.2 Mapeo de Requerimientos vs. Objetos SQL

Requerimiento del Negocio	Componente Técnico	Objeto en el Código
Prohibir registro de menores de 18 años	TRIGGER BEFORE INSERT	trg_cliente_edad_minima
Bloquear gasto sobre el límite diario	TRIGGER BEFORE INSERT	trg_limite_diario
Bloquear clientes autoexcluidos	TRIGGER BEFORE INSERT	trg_limite_diario
Alerta AML en transacciones $\geq$ \$10M	TRIGGER AFTER INSERT	trg_alerta_aml
Impedir sesiones en mesas fuera de servicio	TRIGGER BEFORE INSERT	trg_validar_mesa_activa
Acumular puntos por compra de fichas	TRIGGER AFTER INSERT	trg_acumular_puntos
Calcular premio de torneo (80% del pozo)	TRIGGER BEFORE INS/UPD	trg_calcular_premio_torneo
Auditoría de cambios en datos de clientes	TRIGGER AFTER INS/UPD/DEL	trg_auditoria_cliente
Evitar bloqueos infinitos en caja	ACID - FOR UPDATE NOWAIT	PRAGMA EXCEPTION_INIT(-54)
Espera tolerante antes de timeout	ACID - FOR UPDATE WAIT 5	PRAGMA EXCEPTION_INIT(-30006)
Registrar clientes con validación	STORED PROCEDURE	sp_registrar_cliente
Transacciones de caja con atomicidad	STORED PROCEDURE	sp_registrar_transaccion
Abrir y cerrar sesiones de juego	STORED PROCEDURE	sp_abrir/cerrar_sesion_mesa
Ocultar RTP y rangos al personal	VISTA DE SEGURIDAD	v_catalogo_publico

Requerimiento del Negocio	Componente Técnico	Objeto en el Código
Rentabilidad por sala para la junta	VISTA DE REPORTE	v_reporte_ingresos_sala
Nivel de riesgo de ludopatía	FUNCTION	fn_nivel_riesgo_cliente
Control de acceso por rol	PACKAGE + VISTAS filtradas	pkg_contexto_casino

SECCIÓN 3: MODELADO — DIAGRAMA ER Y DICCIONARIO DE DATOS

3.1 Diagrama Entidad-Relación

El modelo relacional está compuesto por 17 tablas organizadas en tres capas funcionales:

- **Capa Maestra:** cargo, empleado, sala, tipo_juego — catálogos base del negocio.
- **Capa Operativa:** cliente, mesa, maquina, cajero, torneo, inscripcion_torneo, sesion_mesa, sesion_maquina, transaccion, movimiento_puntos — operación en tiempo real.
- **Capa de Control:** incidente, auditoria_log, usuarios_sistema — cumplimiento, trazabilidad y seguridad.

Cardinalidades Principales del Modelo

Relación	Cardinalidad	Descripción
cargo → empleado	1 : N	Un cargo puede tener muchos empleados
empleado → cajero	1 : 1	Un empleado opera una sola caja
cliente → transaccion	1 : N	Un cliente tiene muchas transacciones
cliente → sesion_mesa	1 : N	Un cliente juega en muchas sesiones de mesa
mesa → sesion_mesa	1 : N	Una mesa tiene muchas sesiones a lo largo del tiempo
sala → mesa	1 : N	Una sala contiene múltiples mesas
sala → maquina	1 : N	Una sala contiene múltiples máquinas
torneo → inscripcion_torneo	1 : N	Un torneo tiene muchos inscritos
cliente → movimiento_puntos	1 : N	Un cliente tiene historial de puntos
auditoria_log	N/A	Recibe INSERT automático de triggers de cliente, transaccion y usuarios_sistema

Nota: El diagrama ER gráfico se incluye como docs/diagrama_ER.png en el repositorio GitHub.

3.2 Diccionario de Datos

Tabla: cargo

Columna	Tipo	Restricción	Descripción
cargo_id	NUMBER(6)	PK, IDENTITY	Identificador único del cargo

Columna	Tipo	Restricción	Descripción
nombre	VARCHAR2(100)	NOT NULL	Nombre del cargo (CEO, Croupier, etc.)
salario_base	NUMBER(12,2)	NOT NULL, CHECK > 0	Salario base mensual en COP
descripcion	VARCHAR2(500)	—	Descripción de responsabilidades

Tabla: empleado

Columna	Tipo	Restricción	Descripción
empleado_id	NUMBER(8)	PK, IDENTITY	Identificador único del empleado
cedula	VARCHAR2(20)	NOT NULL, UNIQUE	Documento de identidad
nombres	VARCHAR2(100)	NOT NULL	Nombres del empleado
apellidos	VARCHAR2(100)	NOT NULL	Apellidos del empleado
cargo_id	NUMBER(6)	FK → cargo	Cargo que desempeña
fecha_ingreso	DATE	NOT NULL	Fecha de vinculación laboral
turno	VARCHAR2(10)	CHECK IN (4 valores)	MAÑANA, TARDE, NOCHE, ROTATIVO
activo	NUMBER(1)	DEFAULT 1, CHECK IN(0,1)	Estado laboral activo/inactivo

Tabla: cliente

Columna	Tipo	Restricción	Descripción
cliente_id	NUMBER(10)	PK, IDENTITY	Identificador único del cliente
cedula	VARCHAR2(20)	NOT NULL, UNIQUE	Documento de identidad
fecha_nacimiento	DATE	NOT NULL	Validada por trigger (>= 18 años)
nivel_membresia	VARCHAR2(10)	DEFAULT BRONCE, CHECK	BRONCE / PLATA / ORO / PLATINO / VIP
puntos_lealtad	NUMBER(10)	DEFAULT 0	Puntos acumulados en el programa
autoexcluido	NUMBER(1)	DEFAULT 0, CHECK IN(0,1)	Flag de autoexclusión voluntaria
limite_diario	NUMBER(14,2)	DEFAULT 5000000, CHECK > 0	Tope de gasto diario en COP
gasto_hoy	NUMBER(14,2)	DEFAULT 0, CHECK >= 0	Gasto acumulado en el día actual

Tabla: transaccion

Columna	Tipo	Restricción	Descripción
transaccion_id	NUMBER(15)	PK, IDENTITY	Identificador único

Columna	Tipo	Restricción	Descripción
cliente_id	NUMBER(10)	FK → cliente	Cliente que realiza la operación
cajero_id	NUMBER(6)	FK → cajero	Cajero que procesa la transacción
tipo	VARCHAR2(20)	CHECK IN (5 valores)	COMPRA_FICHAS, CANJE, DEPOSITO, RETIRO, PREMIO
monto	NUMBER(14,2)	CHECK > 0	Valor en COP (no permite cero ni negativo)
metodo_pago	VARCHAR2(20)	CHECK IN (4 valores)	EFFECTIVO, TARJETA_DEBITO, TARJETA_CREDITO, TRANSFERENCIA
referencia	VARCHAR2(80)	—	Código único generado automáticamente por trigger
revertida	NUMBER(1)	DEFAULT 0, CHECK IN(0,1)	Flag de anulación de la transacción

Tabla: auditoria_log

Columna	Tipo	Restricción	Descripción
log_id	NUMBER(15)	PK, IDENTITY	Identificador único del registro
tabla_afectada	VARCHAR2(50)	NOT NULL	Nombre de la tabla modificada
operacion	VARCHAR2(10)	CHECK INSERT/UPDATE/DELETE	Tipo de operación DML registrada
registro_id	NUMBER(15)	—	ID del registro que fue modificado
usuario_bd	VARCHAR2(50)	DEFAULT USER	Usuario de BD que ejecutó la operación
fecha_hora	TIMESTAMP	DEFAULT SYSTIMESTAMP	Momento exacto del evento auditado
detalle	VARCHAR2(1000)	—	Descripción detallada del cambio

SECCIÓN 4: ARQUITECTURA TRANSACCIONAL — JUSTIFICACIÓN ACID

Análisis de Concurrencia — El Peor Escenario Posible

Considerar el siguiente escenario en hora pico un sábado: dos cajeros en cajas diferentes (CAJA-01 y CAJA-02) reciben simultáneamente a dos personas que quieren comprar fichas para el mismo cliente VIP. Si ambas transacciones se ejecutan en paralelo sin control, las dos leerían el mismo valor de gasto_hoy y las dos concluirían que la compra está dentro del límite diario. Ambas se aprobarían, el gasto_hoy quedaría en un valor inconsistente y el cliente habría superado su límite sin que el sistema lo detectara, violando la regulación de juego responsable.

Justificación del NOWAIT (ORA-54) — Operaciones de Caja

En caja de un casino la inmediatez es crítica. Un cajero tiene al cliente de frente en una ventanilla física. Por esto, **FOR UPDATE NOWAIT** es la elección correcta: si el registro ya está bloqueado por otra sesión, el sistema falla inmediatamente con **ORA-54** y el cajero redirige al cliente a otra caja sin espera. El uso de **PRAGMA EXCEPTION_INIT(-54)** en lugar de **WHEN OTHERS** permite dar un mensaje específico y accionable al operador, en lugar de un mensaje genérico que no dice qué hacer.

Justificación del WAIT 5 (ORA-30006) — Verificación de Mesa

El bloque **FOR UPDATE WAIT 5** se usa al verificar disponibilidad de mesa antes de abrir una sesión de juego. Una espera de 5 segundos es razonable porque la operación que mantiene el lock es normalmente muy breve. Se eligieron **5 segundos** porque es el tiempo de timeout estándar de la mayoría de interfaces de usuario antes de mostrar un error. Si el lock no se liberó en 5 segundos, se lanza **ORA-30006** y se notifica al supervisor para reasignar la mesa.

Defensa del Uso de Excepciones Especializadas vs. WHEN OTHERS

- **Mensajes accionables:** **WHEN OTHERS** captura cualquier error incluyendo fallas de hardware. Al capturar específicamente -54 o -30006, el sistema sabe con certeza qué pasó y muestra al operador exactamente qué acción tomar.
- **Relanzamiento correcto de otros errores:** Los errores no relacionados con concurrencia (FK violada, CHECK constraint fallido) se relanzarán con su propio código en lugar de quedar silenciados dentro del handler.
- **Trazabilidad y auditoría:** El **DBMS_OUTPUT** dentro del handler específico registra exactamente el tipo de conflicto, útil para detectar patrones problemáticos de uso concurrente.

SECCIÓN 5: DISEÑO DE SEGURIDAD Y REPORTERÍA

Estrategia de Seguridad — v_catálogo_publico

La vista **v_catálogo_publico** es la interfaz que se expone a clientes en pantallas del casino y a asesores comerciales. Las siguientes columnas fueron excluidas deliberadamente:

apuesta_min y apuesta_max (tabla mesa): Su exposición combinada con el tipo de juego permitiría a jugadores avanzados calcular la ventaja de la casa por mesa y seleccionar sistemáticamente las mesas con menor ventaja para el casino.

porcentaje_pago — RTP (tabla maquina): Es el dato más sensible de las máquinas. Si fuera público, jugadores con conocimiento estadístico seleccionarían exclusivamente las

máquinas con RTP más alto, alterando los ingresos proyectados. La Ley 643 de 2001 exige reportarlo a Coljuegos pero no obliga a publicarlo al jugador.

Pregunta de Negocio Resuelta — v_reporte_ingresos_sala

"¿Cuál sala del casino genera más ganancia neta para la empresa, cuántos clientes únicos atrae, cuál es la ganancia promedio por sesión y cuántas mesas tiene activas? Esta información permite al Director de Operaciones decidir dónde invertir en expansión, qué salas requieren revisión de su mix de juegos y cuál es la eficiencia real de cada piso del casino."

La vista entrega esta respuesta con un único SELECT, sin que la junta necesite cruzar tablas manualmente, y ordena los resultados de mayor a menor ganancia.

SECCIÓN 6: CONCLUSIONES Y BUENAS PRÁCTICAS

Ventajas de Delegar la Lógica al Servidor de Base de Datos

La base de datos es el único punto de control que no se puede eludir. Se identificaron tres escenarios donde delegar la lógica al servidor fue la única solución robusta:

- **Validación de mayoría de edad:** podría haberse implementado en una aplicación web. Pero si alguien conecta directamente con SQL Developer o Python y hace un INSERT directo, la validación de la aplicación no aplica. El trigger `trg_cliente_edad_minima` sí aplica porque vive en el motor.
- **Límite diario de gasto:** un cajero podría ignorar una advertencia en pantalla o usar una herramienta diferente. El trigger `trg_limite_diario` garantiza el límite sin importar desde dónde venga el INSERT.
- **Acumulación de puntos y membresía:** el trigger `trg_acumular_puntos` eliminó la necesidad de escribir esa lógica en cada módulo de la aplicación. El servidor la ejecuta una sola vez, de forma consistente, en todos los casos.

Impacto del Estándar de Nomenclatura en la Integración del Equipo

Trabajar con el estándar definido en la guía (tablas en minúsculas con guion bajo, PKs con el patrón `nombre_tabla_id`, FKs con prefijo `fk_`, triggers con prefijo `trg_`, procedimientos con prefijo `sp_`) demostró su valor al integrar el script final. Cuando los tres integrantes generaron sus bloques de código por separado, las FKs referenciaban correctamente las PKs sin necesidad de revisar el DDL para saber el nombre exacto de cada columna — el nombre mismo lo decía. Esto redujo significativamente los errores de integración al unificar el script final.